

YOLO v11 for Traffic Sign Detection

By Group 1: Adam Rigdon, Leo Grover, and Will Slyman

Rochester Institute of Technology, CSCI 331
December 3rd, 2025

Our Objective

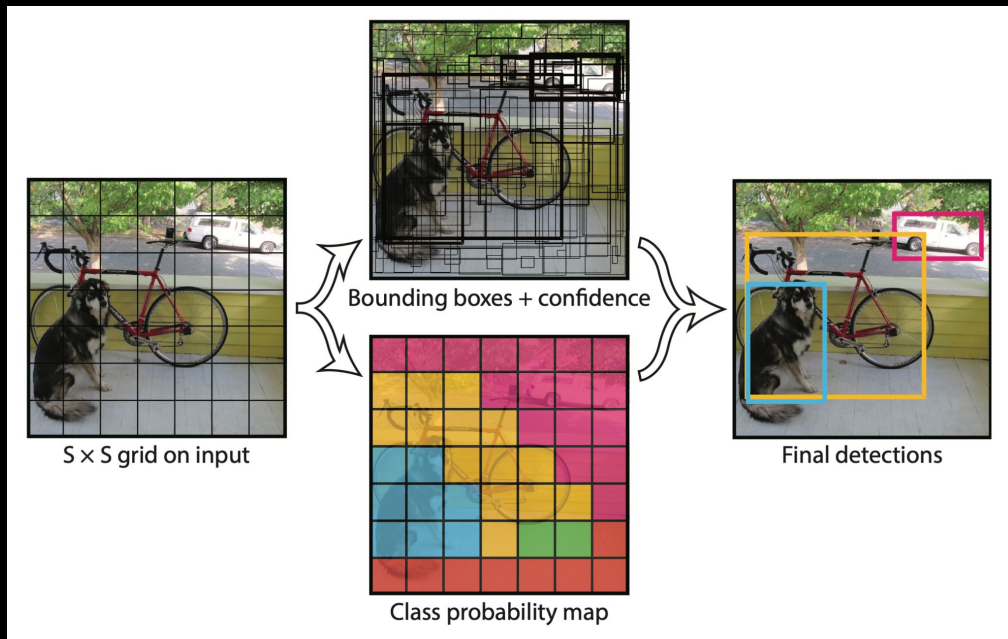
We aimed to develop a system for **identifying** and **classifying** multiple common road traffic signs using a provided dataset.

By combining **computer vision techniques** and **machine learning models**, we believed the system could be accurate even when detecting signs from far away or in various different environments.

Such a system could be used in **self-driving cars** to help them accurately recognize signage, ensuring autonomous vehicles adhere to traffic **rules and regulations** while keeping passengers, pedestrians and other drivers **safe** on the road.

What is YOLO?

"You Only Look Once" (YOLO) is a real-time object detection algorithm that uses a single neural network to process an image and predict bounding boxes for objects.



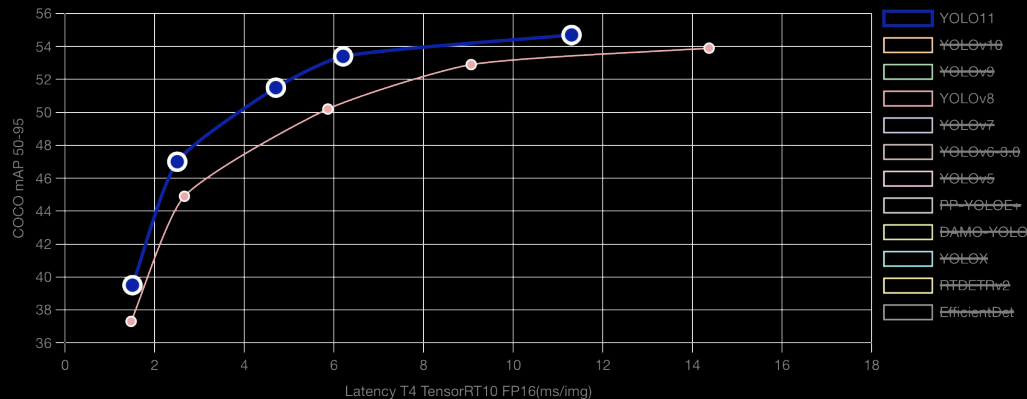
Dataset and Preprocessing

- Classes:
 - Green & Red Lights
 - Speed Limit Signs (10 - 120)
 - Stop Signs
- Dataset Split:
 - Train Set: 3530 images; 71%
 - Valid Set: 801 images; 16%
 - Test Set: 638 images; 13%
 - Total Set: 4969 images
- Preprocessing:
 - Training images scaled to 416x416
 - Predicting based on 640x640 images
 - Images are scaled down to 640 pixels in one direction, then filled in with a set color

YOLO v11 with Ultralytics

We chose to use **Ultralytics YOLO11** in our Traffic Sign Detection System implementation for the following reasons:

- Improved Feature Extraction
- Optimized Speed and Efficiency
- Greater Accuracy with Fewer Parameters



Achieves a **higher** mean Average Precision (mAP) than YOLOv8 while using **22% fewer** parameters

Hyperparameter Tuning Setup

- In order to figure out the best hyperparameters to use, we utilized Ultralytics' **tune** function, with **Ray Tune** enabled for advanced hyperparameter optimization.
 - This tune function utilizes **genetic algorithms** to optimize hyperparameters.
 - **Mutation** is mainly used, while crossover is **not** currently used in the algorithm.

Hyperparameter Tuning Setup

- The hyperparameters we chose to tune were:
 - Learning Rate (`lr0`)
 - Batch Size
 - Number of Epochs

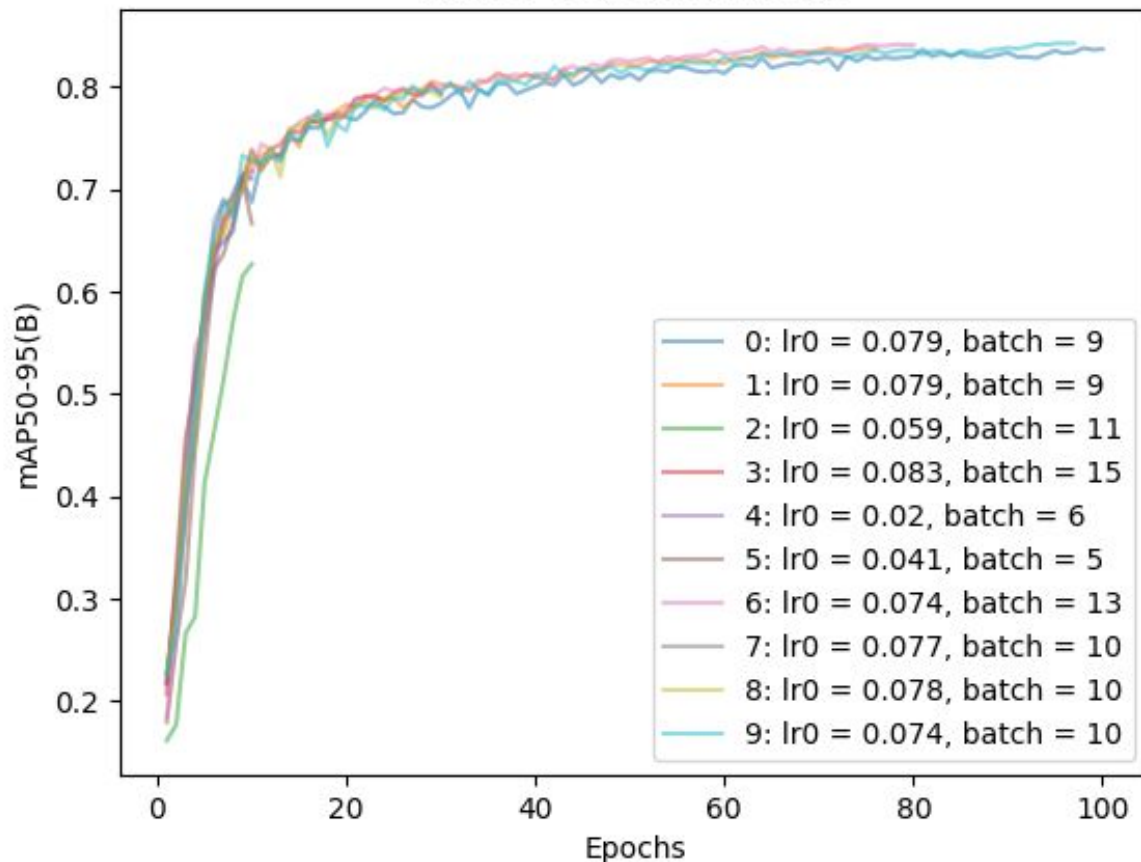
```
# Define search space
# Search for the best values for these hyperparams
# between respective ranges
search_space = {
    "lr0": tune.uniform(1e-5, 1e-1),
    "batch": tune.randint(4, 16),
    "epochs": tune.randint(50, 200)
}
```

Hyperparameter Tuning Setup

```
results = model.tune(  
    data="./code/YOLOv11/conf.yaml",  
    space=search_space,  
    patience=25, # Not a standard hyperparameter  
    imgsz=640,  
    workers=4,  
    name='hyperparameter_tuning',  
    use_ray=True,  
    device=0,  
    gpu_per_trial=1  
)
```

Runs **10 Iterations**,
mutating the learning
rate, batch size, and
number of epochs each
time.

Overall Hyperparameter Tuning Results: mAP50-95(B) v.s. Epochs



mAP50-95

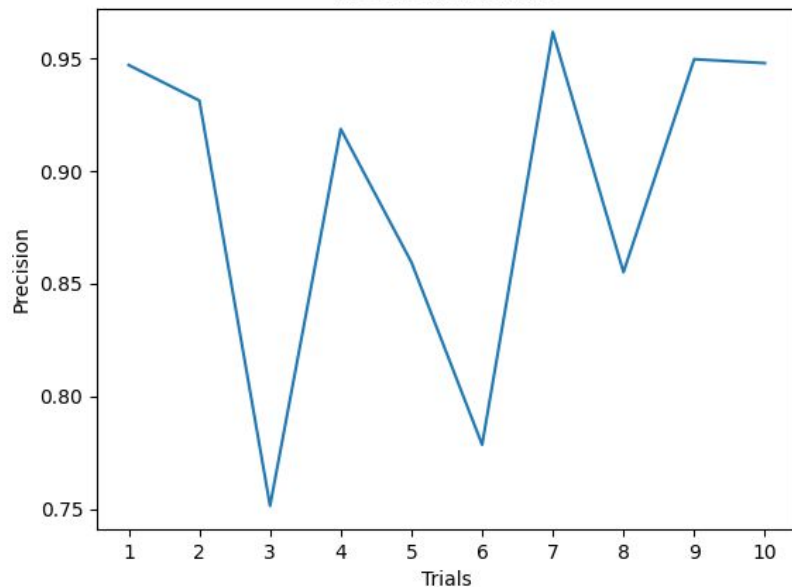
The average of the mean average precision calculated at varying IoU thresholds, ranging from 0.50 to 0.95.

Gives a comprehensive view of the model's performance across different levels of detection difficulty.

(Ultralytics Docs)

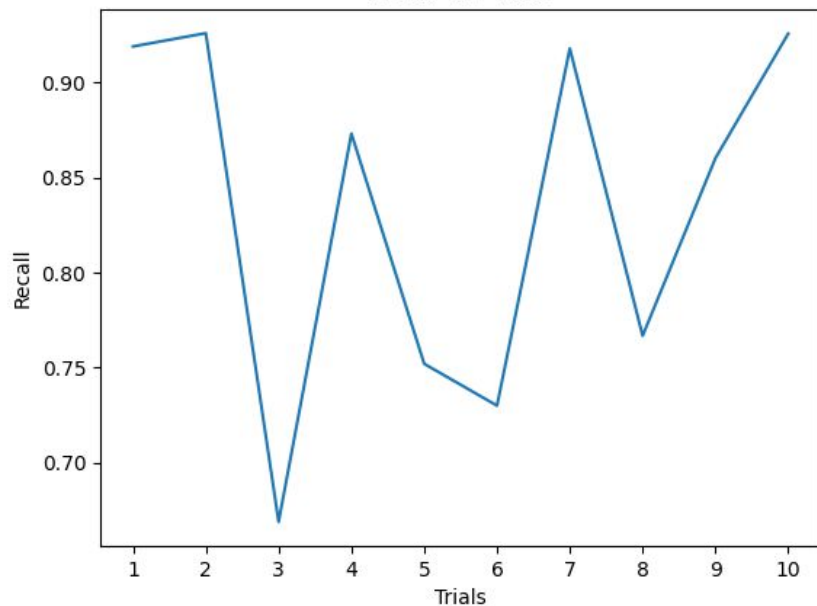
Precision & Recall over Time

Precision v.s. Trials



Change in **Precision**, varies over time but ends with a high score.

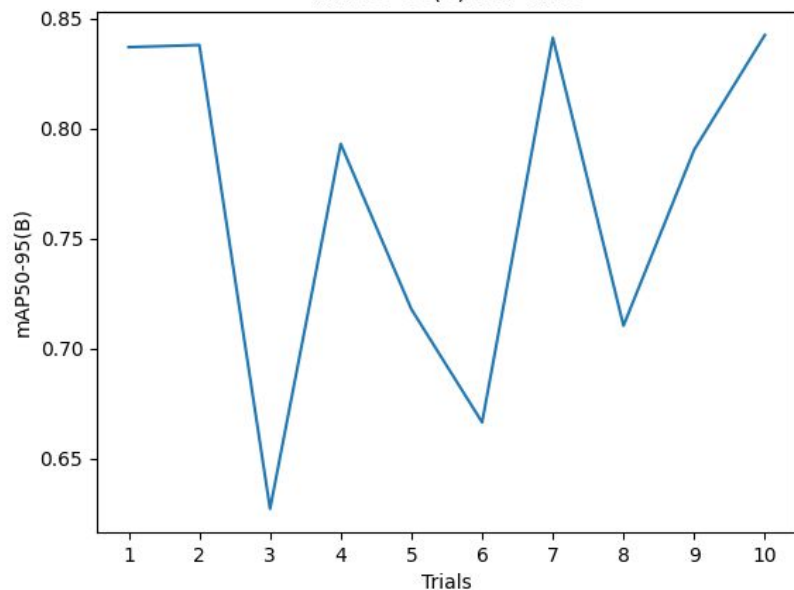
Recall v.s. Trials



Change in **Recall** over time, similarly ends with a high score.

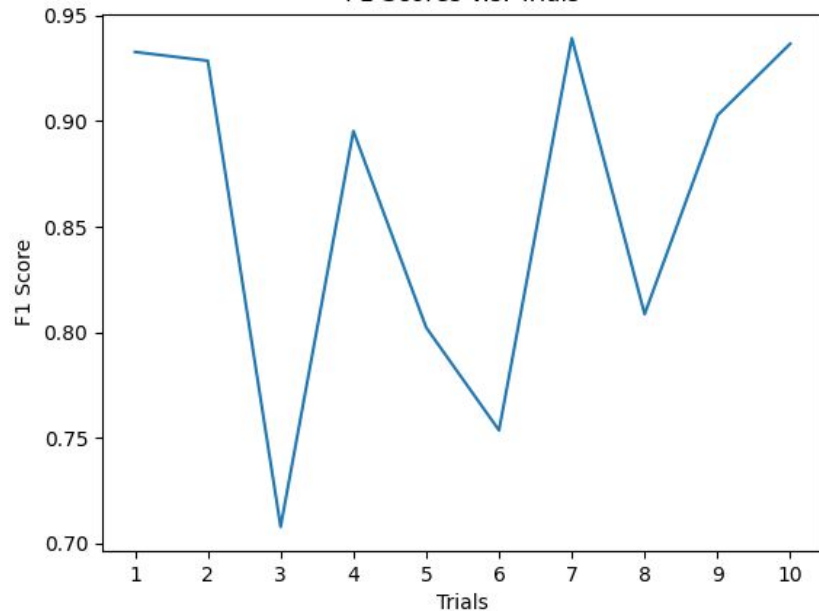
mAP50-95 & F1 Scores over Time

mAP50-95(B) v.s. Trials



Change in **mAP50-95**, varies greatly but ends with the **highest score** so far.

F1 Scores v.s. Trials



Change in **F1 Scores** over time, additionally ends with a high score.

The Best Hyperparameters

(Based on the Maximum mAP50-95 score)

Learning Rate (lr) = 0.07412

Batch Size = 10

Number of Epochs = 95

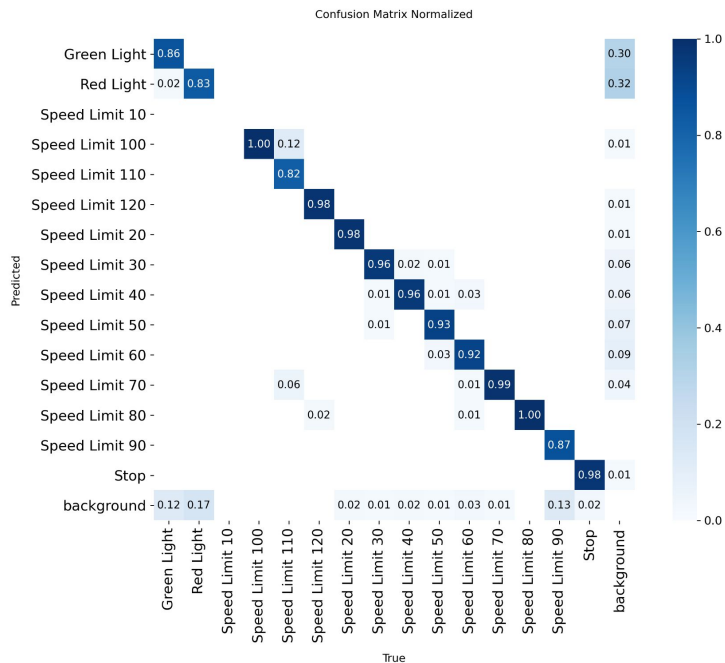
mAP50-95 = 0.8427

Precision = 0.9479

Recall = 0.9257

F1 Score = 0.9367

Results



- The model detects all image types with at least 80% accuracy
- Relatively low confusion with other objects
- Largest confusion between the traffic lights and background, and the 100 and 110 speed limit signs

Results



The trained model can detect the full variety of signs (lights, speed limits, stop signs).

Because of the variation in training data, it can also recognize the signs in poor imaging (like the image with the fisheye lens).

Conclusion

In summary, to build the final system, we **configured** YOLOv11 using the training dataset, **trained** and **validated** the model, and then **evaluated** it on unseen images to measure accuracy and robustness.

Looking forward, the project could be expanded by training on more diverse datasets (like adding the various warning signs, etc), adding sample data from a higher variety of environments (ex: poor vision due to snow), or integrating the detector into a more user-friendly application or end-to-end driving situation.

We chose to train only a single model because the dataset was uniform in style and complexity, and one well-tuned detector was sufficient to achieve strong performance without the overhead of maintaining multiple models. Additionally, there is a significant hindrance with the time it takes to set up and especially train a new model.

Works Cited

Redmon, Joseph, et al. "You Only Look Once: Unified, Real-Time Object Detection." *ArXiv.org*, 8 June 2015,

arxiv.org/abs/1506.02640.

Ultralytics. "Hyperparameter Tuning." *Docs.ultralytics.com*, 2024,

docs.ultralytics.com/guides/hyperparameter-tuning/.

Ultralytics. "Ultralytics YOLO11 Overview." *Ultralytics.com*, 2024,

docs.ultralytics.com/models/yolo11/#performance-metrics.

Ultralytics. "YOLO Performance Metrics." *Docs.ultralytics.com*, 2024,

docs.ultralytics.com/guides/yolo-performance-metrics/#class-wise-metrics.